

LEGO Pipetting Robot

Bachelor Project Report

Student surname(s), first
name(s) and student ID
number

OLIVEIRA DA SILVA Patrick (020152332A)

Name of the study
programme

Bachelor in Applied Information Technology

Names of the instructors

POUSSIN Régine (Scienteens Lab) – professional supervisor
ZAMPUNIERIS Denis (University of Luxembourg) – academic
supervisor

Date of submission

11th June 2023

Abstract

With this paper, I will try to explain to you how I programmed a pipetting robot for the Scienceteens Lab. The pipetting robot is made of Legos. Its users should be able to create artworks on an interface inspired by a biology plate. These artworks are then pipetted by the robot into a small piece of paper.

This project combines biology and computer science. It uses computer science to show, describe, and enhance biology activities. Scienceteens Lab wants to hook high school students into the world of science. Offering them different kinds of workshops and activities is their way of interacting with high school students.

This project has three goals. The first one is to program the robot so that it can complete this task. The second goal is to show how to use computer science to improve the workflow in any work environment. The third goal is trying to show that computer science is not only coding but also and mainly thinking.

Abstract (French)

Avec cette mémoire, je vais essayer de vous expliquer comment j'ai programmé un robot de pipetage pour le Scienteens Lab. Le robot est fait en Lego. L'utilisateur aura accès à une interface inspiré par une plaque de biologie pour créer des dessins. Ces dessins seront une fois terminé pipetés par le robot sur une pièce de papier.

Ce projet combine la biologie et l'informatique. L'informatique est utilisée pour montrer, décrire et améliorer des activités pratiquées dans la biologie. Le Scienteens Lab veut introduire des écoliers au différents mondes des sciences. Des ateliers sur les différents thèmes sont organisés.

Ce projet a trois buts. Le premier, c'est de réaliser la programmation du robot. Le deuxième but c'est de faire que l'informatique peut devenir un atout dans tous les domaines. Le troisième but c'est de montrer que l'informatique n'est pas que coder mais principalement la création d'idées.

Declaration of authorship

For single-authored work:

I hereby declare that I am the sole author of the work entitled

LEGO Pipetting Robot

and here enclosed, and that I have compiled it in my own words, that I have not used any other than the cited sources and aids, and that all parts of this work, which I have adopted from other sources, are acknowledged, and designated as such. I also confirm that this work has not been submitted previously or elsewhere.

08/06/2023

Date



Signature

Patrick OLIVEIRA DA SILVA

First name and surname

Table of Contents

Abstract	2
Abstract (French).....	3
Declaration of authorship	4
Table of Contents	5
Introduction	7
Analysis and Design	10
Lego Robot.....	11
Motors	12
EV3Brick.....	13
Painting Application	15
Explanation Application.....	17
Implementation	18
Tools.....	19
Application Interface Part.....	20
Libraries	21
Window Creation.....	21
Canvas.....	22
Drawing Fields and Colour Palette	23
Scienteens Lab Logo	26
Buttons	26
Cursor	27
Mouse events	27
Conclusion	31
Application Computational Part.....	32
Libraries	32
Creating painting matrix for backend.....	32
Saving and Loading	33
Creating output files	34
Sending files to robot	36

Robot.....	39
Movement	39
Listening & Reacting	43
Explanation Application.....	45
<i>Testing and Validation</i>	<i>46</i>
Accomplished.....	47
Not Done Yet	49
<i>Conclusion.....</i>	<i>50</i>
Improvements for the future	51
Some words about the internship	52
Doing the project from scratch	53
Appreciation words.....	54
<i>Table of Figures.....</i>	<i>55</i>
<i>Bibliography</i>	<i>56</i>
<i>Appendix – Project Reports.....</i>	<i>57</i>

Introduction

Students have lots of different career opportunities at school and after their secondary degree. For every one of them, the first step on what is to become the rest of their lives. Taking that decision based on what has solely been learned at school may seem easy, as students just tend to pick the subjects, they are good at. Personally, I think that the experience I gathered at school does not reflect well what I am studying at university. Friends who already work also tend to tell the same about work. I am sure that many students fall in the same trap of choosing the subject they are best at without knowing what it means to work in that domain. The Scienceteens Lab offers students the possibility to discover more on the different fields of science.

‘The Scienceteens Lab – De Labo fir Jonker – is the first research lab for high-school students in Luxembourg. It is an extracurricular learning centre of the University of Luxembourg that offers workshops designed to spark their interest in science, show them the latest trends and technologies in research and supports them in their career choice. The Scienceteens Lab welcomes classes from all over the Greater Region.’ [1]



Figure 1 Scienceteens Logo

Students can engage with the Scienceteens Lab by participating in their workshops hosted in the University of Luxembourg or during public scientific events. These workshops are meant to show students how working in the different fields of science (biology, physics, mathematics, and computer science) may look like. The workshops are also a way of igniting the interest of students who maybe thought that a specific subject seemed boring at school. Like previously stated, school does not reflect well what is and can be done in the different fields of science. However, students need to start with the basics taught at school. There is no other way around. Scienceteens Lab's workshops take those basics and apply them to show how important they are and what can be done at an advanced level thanks to those basics. Showing the students that science is way more interesting when one invests the time to understand and experiment with it.

In biology and chemistry, experiments often expect the usage of pipettes. Taking the right amount of the right substance, putting it on the right place and so on. Tasks without intellectual value. Today, with technology these tasks could be automated by a computer.

Why may you ask? Well, these repetitive tasks that cannot be skipped do not bring any value to the research. Not only this, but these tasks are also time-consuming and exhausting because they often still need a certain level of concentration to be well done. You do not want to do them half-hearted as doing so may cause your whole experiment to fail and you will probably have to start over. Imagine having a robot doing these tasks for you. The robot is faster and more consistent than you. You just need to get it to work once, and it will be able to repeat that task until its robot heart stops. The researcher wins not only time by having this robot, but also keeps his mind fresh for the research.

These kinds of tasks could be done by computers, and humans would just need to reprogram the robots and make the observations to conduct the experiments. The probability of making human errors would be diminished by a great amount.

Dr. Philip Goergen of the biology department possesses a liquid-handling robot made of Legos. This robot will be addressed as pipetting robot starting here. That robot mimics the pipetting activity. It picks up colour with the attached pipette and puts colour where it has been asked to. Dr. Goergen wants to offer to the public an interface where people can draw on a grid of the same size as the pipetting plates used in biology. The drawing realised on the interface can then be reproduced by the Lego robot.

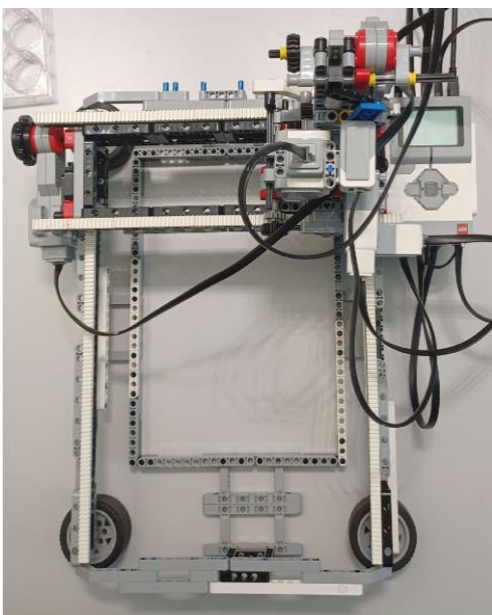


Figure 3 Pipetting Robot

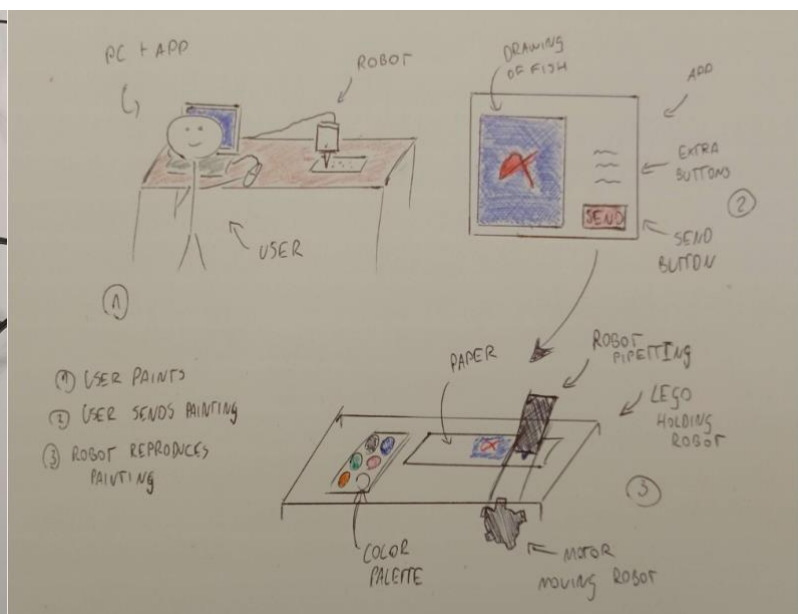


Figure 2 Imagination of how the program will run

This will allow its users to discover what kinds of processes in biology can be automated with computer science and how computer science can be used outside of the computer science field. They can learn as much of both scientific fields by trying the robot out. They can learn about the biology aspect respected in this project and/or the computer science behind the robot's implementation.

Alongside the robot and its painting application, an additional application is necessary. In this application, called explanation application, the robot's way of pipetting will be shown and explained. This will be useful for public events. People can read and try to understand the robot's logic alone and then if interested ask further questions. This will not only help them but also people that are a bit shy to directly ask as they can simply read for themselves. This will also be a big aid for the staff holding the workshop. During such events, the staff must deal with lots of visitors and every help is welcome.

The only thing I have to my disposal is the robot. Nothing to work with has been shared. No documents to use, no guidelines, and no technical information on the robot. I had to find everything by myself as there was no other person working in this project. The Scienceteens Lab team was available for questions, ideas, and opinions.

The team consists of the three head biologists, Dr. Goergen Philip, Dr. John Elisabeth, and Dr. Krüger Maren. The head computer scientist and my professional supervisor, Poussin Régine, and the web designer Tetiana Melnychuk. I exchanged with these people in a daily basis about everything this project offers.

Analysis and Design

In this chapter, the different components of this project will be analysed. After analysing the different components, tools to help implementing this project will be picked.

Lego Robot

The pipetting robot's brain is its micro-computer the EV3Brick. With the help of motors and sensors, the robot can be moved around.

The pipetting robot made of Legos being worked with has been constructed prior the start of this project. Some projects have been already realised by members of the Scienceteens Lab for similar robots. Robots build with Lego Mindstorm sets are usually programmed with the Lego developed software: EV3Classroom. However, in this software programming blocks are used. Programming with programming blocks has its limitations. That is why the robot is programmed with a programming language. The idea is to use a programming language to program the robot in a way that it can react to any kind of requests it gets. The request it will get being the paintings realised on the application.

Before analysing the technical aspects, let's analyse the Lego robot. The following dimensions are approximate values just to help you visualise its size. The robot is around 40cm long, 30cm wide and its height varies between 20cm and 30cm depending on the pipette's current height level. The robot is quite big. Comparing the robot's size to the size of a shoebox is not far-fetched. Not something you can simply put in your backpack.

The robot has been analysed before starting to develop the different programs. However, other limitations have been discovered during implementation. Some of these limitations will be explained in the implementation chapter to help you understand why some common things in programming are being done in an unusual way.

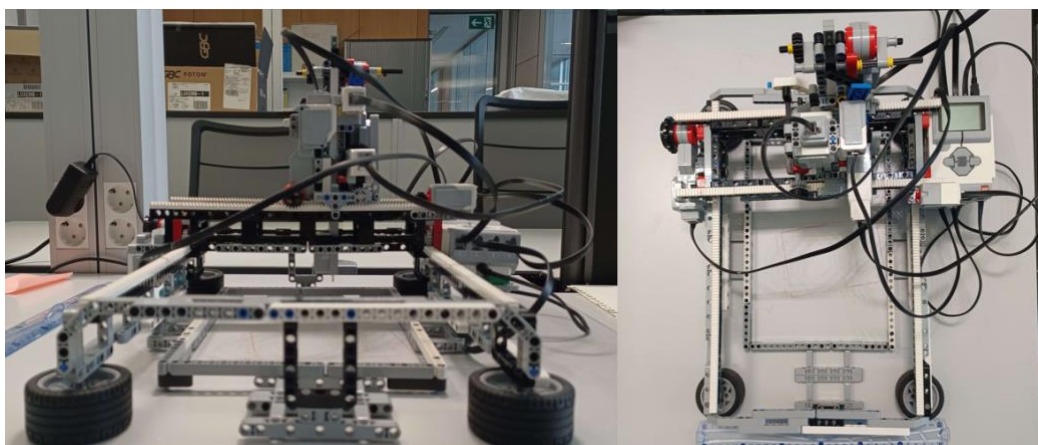


Figure 4 Front and top view of the pipetting robot

Motors

The motors move the robot around in its field of activity. Sensors help setup the robot to always start in the same position.

In Figure 5, the different motors and sensors are displayed. A colour has been attributed to each motor. Same coloured sensors are used together with the motors. The motors will move the robot around and the sensors will be used like brakes. The sensors are supposed to stop the motors from trying to move somewhere outside of the robot's field of activity.

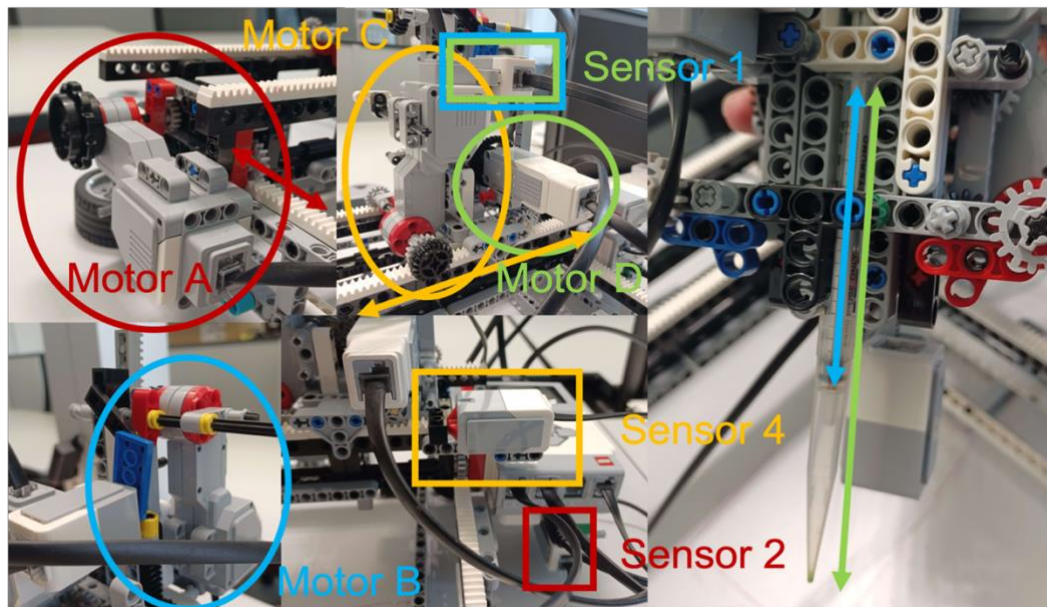


Figure 5 Robot's motors (circles), sensors (rectangles), and movement directions (arrows)

The motors are named after the port on the EV3Brick they are connected to. Port number 3 is not being used and that's the reason why in Figure 5 there is no Sensor 3. Sensor 1 having two colours is intended as it is being used by two motors: Motor B and Motor D. Inside the pipette is a syringe allowing the pipette to be filled and emptied.

Table 1 Pipetting Robot's motors and sensors

	Colour	Moves ...
Motor A	Red	Robot along length
Motor B	Blue	Syringe up and down
Motor C	Yellow	Robot along width
Motor D	Green	Pipette up and down

	Colour	Type of sensor
Sensor 1	Blue Green	Colour sensor
Sensor 2	Red	Touch sensor
Sensor 4	Yellow	Touch sensor

The robot has at specific places different coloured pieces attached to be used with the colour sensor.

EV3Brick

Now, (most of) the robot's toolbox has been presented and explained. Let's go to it's main tool not yet shown, the EV3Brick. The EV3Brick is the central unit of the robot. The EV3Brick is a micro-computer similar to an Arduino or a Raspberry Pi, but developed to be used with robots made of Legos. The EV3Brick is the robot's brain. This part is connected to the PC where the drawing application will be running. The connection is done per USB. The motors and sensors are all connected to the EV3Brick by cable. Some cable management is necessary for the robot to not be hindered while moving.



Figure 6 EV3Brick

Now, the first problem appears. How to run a program not produced on the EV3 Classroom software on the EV3Brick? Well, while reading the [documentation](#) I discovered that you cannot simply run a program not created with EV3 Classroom on your brick. First, the EV3Brick needs to be equipped with a microSD card. This microSD card needs to have a minimum capacity of 4GB and a maximum capacity of 32GB. A card with Application Performance A1 is recommended. [3]

Next, the [EV3 MicroPython card image](#) needs to be installed in that microSD card. The software [Etcher](#) completes this task completely fine. This image is a Lego-approved version of ev3dev. Ev3dev is a Debian Linux-based operating system. This operating system works with the EV3Brick but also other micro-computers like BrickPi. BrickPi is a micro-computer like the EV3Brick, but Raspberry Pi-powered allowing it to have more functionalities. [4]

After installing the image in your microSD card, insert it in the EV3Brick. Now, that the microSD is installed and the EV3Brick is booted, the connection is possible. An extension for EV3 Micropython has been created for Visual Studio Code. This extension allows connecting to the EV3Brick with that editor and it also allows running the written programs on the robot.

Micropython has a library called pybricks. This library contains many features to manipulate the robot. Displaying some text on the screen, rotating a motor by x-degrees, and using the speaker are some of the manipulations that can be done with this library.

The EV3Brick can only run one Micropython program at a time. This means that only one script will be developed for the robot. That script must be able to “receive” and follow the instructions to draw the paintings realised on the painting application. More explanations to this will follow on the implementation part.

Painting Application

The painting application should be appealing and ergonomic for its user. It is important to have an idea of what needs to be done but still be flexible enough to adopt new ideas.

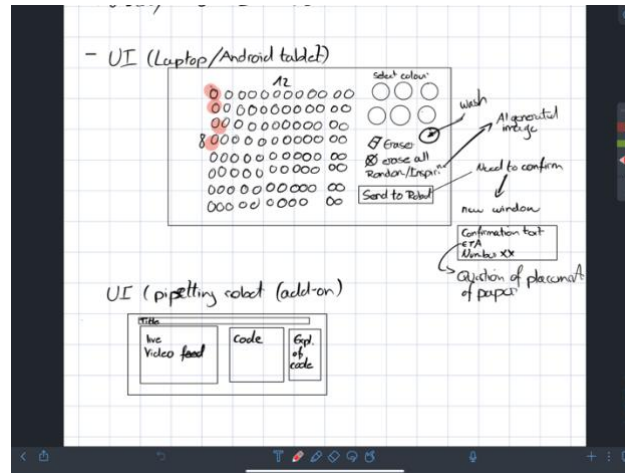


Figure 7 Initial Design Idea

For the painting application, Dr. Goergen drew what the application should look on his tablet. In Figure 7, the initial design idea is displayed. In biology, plates of the size 8x12 (8 columns and 12 rows) are used in a lot of experiments. The robot will simulate a procedure done in biology. Users will have to paint in such a plate on the computer. This plate can be found on the left side of the design. This was the initial idea. However, the robot cannot reach the level of precision necessary to use the plates due to it being made of Legos. The grid is increased to 10x16. On the right, we have 6 colours in a 3x2 grid and some buttons. The size of the colour palette is 3x2 because the plate, on which the colours are, has a size of 3x2. The colour palette's size will also change. The final size is 1x6. The user must be able to choose the colour (s)he desires by clicking on the desired colour. Then paint on the left grid by either clicking or by dragging the mouse on the different spots. Next, the application has four additional buttons. The first button enables the eraser and allows erasing already drawn spots. The second button clears the drawing grid. Useful to start a new painting. The generate button loads a random pre-drawn picture into the drawing grid. These pictures are drawn by the Scienceteens Lab. The last button sends the necessary information about the drawn painting for the robot to replicate it.

This application also has some features not visible on the design idea. One of them is changing the mouse cursor into a brush while using this application. The tip of the brush should change with the selected colour and serve as colour indicator. When the eraser is enabled, the mouse cursor should be an eraser. When a painting is done, that painting must be stored. When the send button is pressed, a confirmation window should appear. In that confirmation

window a code will be given to the user. The code is useful to pick up the painting done by the robot. Keep in mind, that the robot will have a queue and the paintings will need to dry before the user picks theirs up.

Micropython is very similar to Python and Python has some libraries that can create what has just been described. These reasons will lead to the application window being written in Python. I choose Python because I have not done any major Python project yet and I want to enrich my Python knowledge. I also thought that there would maybe be compatibility issues between Micropython and another programming language, but it does not really matter as they are not directly communicating with each other. In Python, there is a library called tkinter. Tkinter is the standard Python library to develop graphical user interfaces (GUIs). Tkinter allows drawing, creating buttons, changing the cursor, and much more. Tkinter is perfect for this task. Tkinter was picked because it has everything this application needs and because it has good documentation and a big online community.

For this application, two scripts will be written. The first script will create the application window and its widgets (buttons, colour palette and drawing fields). The second script will run the backend of the application.

The project management methodology for this project is very similar to an Agile methodology. Every time, a new feature was developed, or the layout changed, a short meeting took place. The robot is sometimes a bit cryptic, so it is difficult to know in advance what needs to be done. These short meetings allowed adding and dropping features without it causing too much trouble or any additional work.

The project has been developed on a Mac operating system. To make it more accessible, the application also runs on Raspbian. Raspbian is an operating system developed for Raspberry Pi. Making this project executable on a Windows operating system would be ideal. It is possible to run the user application alone. It is also possible to start the robot with the help of a Visual Studio Code extension. However, doing both together does not work, because no libraries have been found that allow connecting the robot and the user application automatically.

Explanation Application

The explanation application is a separate application intended to run on an additional screen next to the robot. In this application, animations and text are used to explain how the robot is working.

In this application, the robot's movement will be explained step-by-step. The application shows the drawing being realised and below it is a drawing area like on the user application. The drawing area will be filled in the same order as the robot is drawing. Next to that drawing area, the spectators can read what instructions are being executed. Additional text explains the code the robot is executing without showing the actual code. The different steps are explained in text and the actual instructions the robot receives from the painting application are displayed along that text.

This application's design will be very similar to the user application's design. If everything goes as planned, most code will be reusable. Most code will only need to be adapted. This application will be developed at the end.

The design of this application differs from the one shown in Figure 7. No live feed is implemented into this application as the robot will be running next to the display on which this application is running.

Implementation

In this chapter, the code written, and the algorithms created will be explained here. Please refer to the library documentation if you need more information about certain python commands. Documentation links to the different libraries used will be provided shortly.

This part will be split in five parts. First, the tools used will be presented. The second part will cover the interface of the application. The interface consists of everything the user can see and feel. The third part will cover the backend of the application. This means everything the user cannot see. The fourth part will cover the algorithms created to manipulate the robot. Finally, the last part will cover additional scripts developed for this project that do not belong to the before mentioned categories.

Tools

- Visual Studio Code. VSC is a code editor. VSC supports most programming languages. Many extensions are available on VSC allowing to develop all kinds of projects.
- Sonarlint. Sonarlint is like a spell checker for programming languages. Sonarlint is compatible with VSC.
- LEGO® MINDSTORMS® EV3 MicroPython. This extension allows connecting your pc to the EV3Brick. With this extension the content of the EV3Brick can be seen and on Visual Studio Code developed programs can be sent and run by the brick. This extension is compatible with VSC.
- Python. Python is the main programming language of this project. Python can run Python and Micropython programs.
- Raspberry Pi 3B v1.2. A Raspberry Pi is a microcomputer. A Raspbian image with desktop functionalities has been installed on the Raspberry Pi.

Application Interface Part

This chapter covers the interface of the painting application. Most code used for the interface is about its look-and-feel.

Before starting with the implementation of the interface, some words about the interface's development. The interface had during development different designs. During exchanges, new ideas were born and explored. Some stayed, some not. The procedure of creation was the same for all of them. Functionalities did change and so did the code. Following this are two pictures that show what the application looked like during different stages of development.

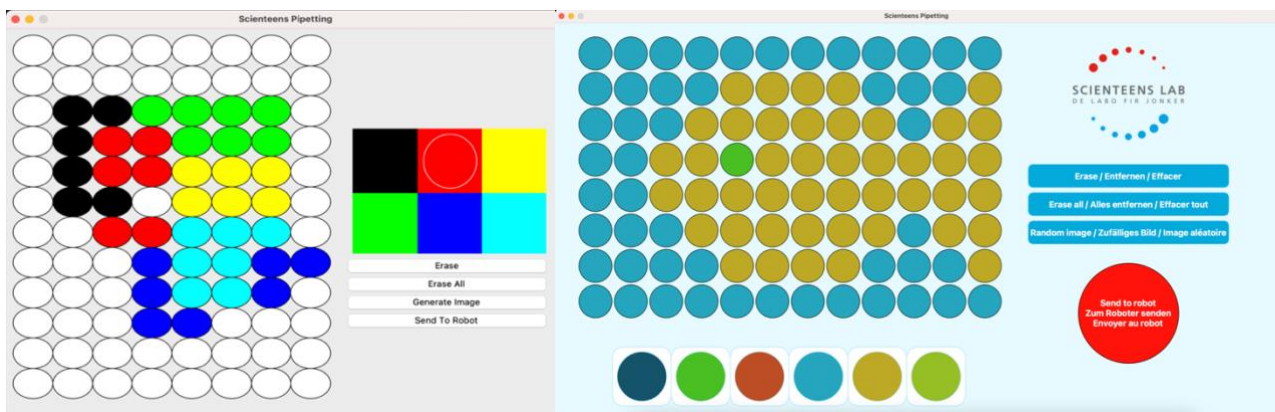


Figure 8 Different designs of the application

These changes happened after exchanging with other team members. The first version is highly inspired by the sketch drawing shown earlier (Figure 7). The design had to be changed to fit more with previous projects of the Scienceteens Lab but also to be more appealing to the human eye. Many ideas were suggested and discussed. After, every change opinions were exchanged. The colours, the font, the spacing, and other details had to be changed. Scienceteens Lab's web designer, Tetiana Melnychuk, helped improving the interface. She gave advice on the design. Tetiana participated in a lot of other projects and gathered knowledge on user interaction during these. Combining the different members' ideas and Tetiana's advice were crucial for the front-ends creation.

This application has a version for Mac and one for Raspbian. The differences between both are minimal as both operating systems run on Unix. The main difference between the two comes down to the measures used for certain dimensions and that the cursors are only available on Mac.

Libraries Used

- Pillow allows processing images with Python.
Documentation: <https://pillow.readthedocs.io/en/stable/>
- Tkinter offers a GUI toolkit compatible with Python.
Documentation: <https://docs.python.org/3/library/tkinter.html>

Window Creation

Creating a Tkinter application can be done with some basic code. Setting it up is in no way more difficult. Just keep in mind that everything must be programmed.

For the interface, only the tkinter library has been used. To create a tkinter application, three lines of code are needed.

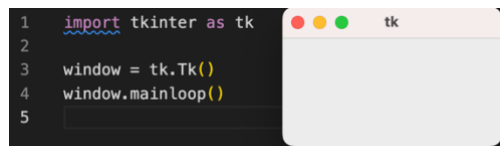


Figure 9 Basic tkinter application

The first line imports the tkinter library into the script we are working on. Line 3 creates an instance of the Tk class. This class has been defined in the tkinter library and offers lots of tools for GUI manipulation. Line 4 keeps the application window alive and running. Without this line, the script would execute the code written and close the application once it is done. This would maybe be so fast that it would seem as if nothing had happened. This three lines are a must for any kind of tkinter application.

In Python, applications can be programmed without the use of classes, but I prefer working with classes. So, a class called UI (user interface) will be created. This class will be responsible for the front-end part of this application. Front-end includes the application's design and ergonomic. In other words, front-end is everything the user sees and feels.

After creating the window, some configurations must be done. The dimensions of the application window must be set. The application needs to run on fullscreen. The application must not be resizable as this may cause the layout to change. The title of the application window should not be "Tk" but something more fitting. Finally, the "update()" function needs to be called because the dimensions have been changed and I want to re-use the applied values without having to create new attributes if possible. Calling this function is not necessary to see the changes, but is necessary to change internal values used by other Tkinter functions.

Canvas

In this canvas, the application elements like buttons and drawing area will be placed and created.

In Tkinter, you need a canvas if you want to create shapes on your application window. Here, many shapes need to be created and managed. In Figure 7, you can see the different shapes that will be necessary. The drawing area consists of many circles. The colour palette and the buttons are other elements that need to be placed on the canvas. These shapes are called widgets. Some widgets like buttons or labels can be placed on the root of the application window. However, geometry widgets like rectangles and circles need to be placed on a canvas which is placed on the root. A canvas is also a widget. Each widget needs to have a parent to be placed on. The canvas' parent is the application window. The other widgets' parent will be the canvas.

Creating a canvas is very simple and like creating a window. Everything needs to be configured. The height, the width, the background colour and if necessary, more values can be configured.

The canvas needs to cover the whole application window. Here it is important that the "update()" function has been called because now we can take the root window and just use the dimension applied to it without ever having known the exact measure. Next, the Scienceteens Lab baby blue colour is applied on the background. Finally, the border that is given by default to the canvas is removed.

```
1  import tkinter as tk
2
3  class UI:
4
5      def __init__(self):
6          # default, background, outline, button, active button, send button
7          self.design_colors = ["#FFFFFF", "#E6FAFF", "#A0E1FF", "#05AADC", "#9DCEC7", "#FF140B"]
8
9          self.window = tk.Tk()
10         self.canvas = tk.Canvas(master=self.window)
11         self.setup_application_window()
12         self.window.mainloop()
13
14     def setup_application_window(self):
15         """configures the application window and its canvas"""
16         self.window.wm_attributes('-fullscreen','true')
17         self.window.resizable(width=False, height=False)
18         self.window.title(string="Scienceteens Pipetting")
19         self.window.update()
20         self.canvas.configure(width=self.window.winfo_width(), height=self.window.winfo_height())
21         self.canvas.configure(highlightthickness=0)
22         self.canvas.configure(bg=self.design_colors[1])
23         self.canvas.pack()
24         self.window.update()
25
26 ui = UI()
27
```

Figure 10 Setting up the application window and its canvas

In Figure 10, you can see the different lines of code allowing to do what has just been explained. Most lines of code are straightforward. In line 21, “highlightthickness” is the default border given to the canvas that needs to be removed. Additionally, the “pack()” function places the canvas on the root window and turns it visible for us.

In line 7, you see a list with colours stored in it. These colours are used for designing the application window. To save some lines, the different colours are stored in a list. This kind of storing values will be used several times.

Drawing Fields and Colour Palette

The drawing fields and the colour palette are created with geometry widgets. Remember, geometry widgets must be placed on a canvas.

Before starting with drawing, some values need to be stored. To save some lines, they will be stored in a list. First, the smaller size between height and width must be figured out. The smaller size is usually the height. This project is intended for horizontal screens. Next, the number of rows and columns for the drawing fields must be stored. At first, the plan was to use an 8x12 grid because the 96 PCR plates used in biology have a size of 8x12. Due to limitations on the robot’s side, the grid size is increased to 10x16. These three values are the main values that need to be stored. Some additional values for layout like margins and paddings are stored.

```

5      def __init__(self):
6          # default, background, outline, button, active button, send button
7          self.design_colors = ["#FFFFFF", "#E6FAFF", "#A0E1FF", "#05AADC", "#9DCEC7", "#FF140B"]
8          # red, orange, yellow, green, blue, black
9          self.color_palette = ["#E84427", "#FF801C", "#F5EC1B", "#477D32", "#246CEA", "#000000"]
10         # cols of draw grid, rows of draw grid, cols of color palette, rows of color palette
11         self.dimensions = [16, 10, 6, 1]
12         self.tags = ["field", "color", "clr_bkg", "eraser", "erase_all",
13                     "generate", "send", "save", "load", "text"]
14
15         self.window = tk.Tk()
16         self.canvas = tk.Canvas(master=self.window)
17         self.setup_application_window()
18
19         # minimum dimension, vertical margin, left margin, right margin, logo size, font
20         self.layout = [min(self.canvas.winfo_width(), self.canvas.winfo_height()),
21                        int(0.06 * self.canvas.winfo_height()),
22                        int(0.03 * self.canvas.winfo_width()),
23                        int(0.125 * self.canvas.winfo_width()),
24                        -1, ("Engel", 18, "bold")]
25         self.layout[4] = int(0.23 * self.layout[0])

```

Figure 11 “__init__” method before creating drawing fields and colour palette

Next, some tags are created. Most widgets will be categorised with the help of tags. For now, they are not important. Finally, the colours of the colour palette are stored in a list. However, unlike previous lists this list is not only here to save some lines but to also ease the access on the different colours. Later, you will see some algorithms that benefit from the colours being in a list and not stored individually.

In Figure 11, the updated “__init__” method is shown. This method is executed once when a new instance of this class is created. It is good to keep this method the place where the different attributes are defined because if something needs to be changed, you know where you need to apply the changes. It also turns the code easier to read.

Next, a method that creates the drawing fields and the colour palette is defined. Both are created in the same method as one should not exist without the other and their creation procedure is very similar.

In this method, some mathematics are used. The size of one field needs to be calculated. To get the size, the minimum window dimension (height or width) needs to be divided by the number of columns or rows. Whoever is the highest. Next, we define some padding, and we locate the margin values defined earlier.

To draw the drawing area, repeat the following one time for each row. Define the start and end y-coordinates of the current row. The y-coordinates of each field in a row remain the same. After defining the y-coordinates of the current row draw one field for each column the drawing area has. The x-coordinates need to be calculated for each field.

To calculate the start coordinates you need to follow the following steps:

- Multiply the current row / column with the length of one field.
- Add the margin and the padding to the previous result.

To calculate the end coordinates you just need to add the length of one field and subtract the padding from the start coordinates.

The margin keeps the drawing area from the window border away and the padding prevents the fields from touching each other.

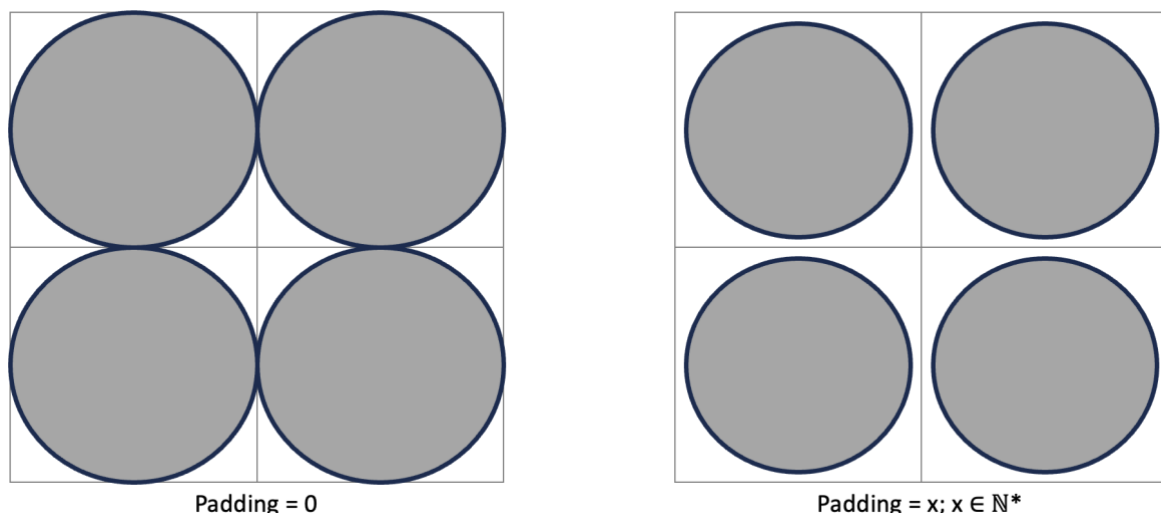


Figure 12 Padding in the drawing fields

To create the colour palette, the same procedure is applied. The colour palette will not be as wide as the draw area. Because of this, the colour palette will be centred below the draw area. To make this happen, the length of one draw field is multiplied by the number of columns and the result of this multiplication is stored.

Next, the size of one colour field is defined and the width of the colour palette is calculated the same way the width of the draw area was. Now subtract the colour palette's width from the draw area's width and divide it by two to get the start x-coordinate of your colour palette. The colour palette is centred.

The colour fields are created the same way the draw fields were. However, the colour palette only has one row, so the y-coordinates just need to be calculated once. The rest remains the same.

The colour palette needs to look like a real colour palette. So, backgrounds have been added to the colour fields. The backgrounds are rounded rectangles. Surprisingly, Tkinter does not offer a built-in method to create rounded rectangles. To create them, a polygon needs to be created instead. This solution was found on a Stack Overflow [forum](#). I adapted the solution to fit with this application.

For it to look nice, layout values have been adapted and added to both grids. Tags have also been added to the different widgets during their creation.

```

77 def create_art_area(self):
78     """creates the draw grid and the color palette"""
79     padding = 4
80     side_length = self.layout[0] / max(self.dimensions[0], self.dimensions[1])
81     for row in range(self.dimensions[1]):
82         start_y = row * side_length + self.layout[1] + padding + 15
83         end_y = start_y + side_length - padding
84         for col in range(self.dimensions[0]):
85             start_x = col * side_length + self.layout[2] + padding
86             end_x = start_x + side_length - padding
87             self.canvas.create_oval(start_x, start_y, end_x, end_y,
88                                   tags=self.tags[0], fill=self.design_colors[0])
89
90     draw_grid_width = self.dimensions[0] * side_length
91     side_length = int(0.15 * self.layout[0])
92     padding = 10
93     palette_width = self.dimensions[2] * side_length
94     left_mar = int((self.layout[2] + (draw_grid_width - palette_width) / 2))
95     start_y = self.canvas.winfo_height() - self.layout[1] - side_length * 1.2
96     end_y = start_y + side_length
97     for col in range(self.dimensions[2]):
98         start_x = col * side_length + left_mar
99         end_x = start_x + side_length
100        coords = [start_x, start_y, end_x, end_y]
101        self.create_round_rect(coords=coords, tag=self.tags[2], radius=50)
102        field = self.canvas.create_oval(start_x+padding, start_y+padding,
103                                       end_x-padding, end_y-padding)
104        self.canvas.itemconfig(field, fill=self.color_palette[col],
105                              outline=self.design_colors[2])
106        self.canvas.itemconfig(field, tag=self.tags[1])

```

Figure 13 "create_art_area" method

Scienteens Lab Logo

The Scienteens Lab logo must appear in the user application. It will appear in the upper right corner.

First, the PIL library is imported. Remember, the PIL library, also known as Pillow, allows manipulating images in Python. Two modules of the PIL library will be used. Modules can be imagined as sub libraries. For the first part the “Image” module is used. We use the “open()” function of the PIL library to open our image. Next, the image is resized. The image is as wide as high. Next, the “ImageTk” module is used. This module allows using images with Tkinter. The resized image is converted into an image that Tkinter can use. That image needs to be stored in an attribute or else the loaded image will be destroyed, and garbage collected.

Buttons

Tkinter offers methods to create buttons. However, customisation does not work well on Mac. So, custom buttons have been created.

The buttons are rounded rectangles with text on them. The size of each button is the same. The last button is an exception. This button is completely round, and its colour is different. This is intended to make the user realise that that button is (s)he should press.

To create a button, a method is created. This method prepares a template of a button. The parameters of that method are then applied on that template. The parameters ask the row, the text, and the tag(s) of the button.

A second method is created. This method makes use of the first one to create the different buttons.

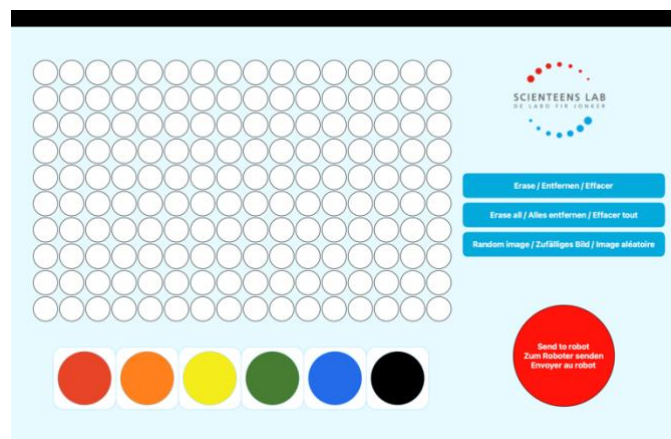


Figure 14 Final UI application design

Cursor

This application will use custom cursors. Users of this application will be able to paint with different colours and to erase parts of their painting with an eraser. The cursor is replaced with a brush whose tip is of the same colour as the colour being used. When the eraser is enabled, the cursor becomes an eraser.

In Tkinter, the cursor is a property of the application window. Changing the cursor is therefore simple. The cursor is just another value like width, height, or background colour.

The custom cursors have been drawn by Dr. Goergen. The cursors are .png files. The hotspot of a cursor is the part of the cursor that clicks. Normally, the hotspot is on the top left of the cursor. Somehow, Tkinter is not placing the hotspot on the top left but somewhere else. The reason for this has not been found. However, a work around has been found. The hotspot has not been fixed but when someone clicks with a custom cursor, the application reacts not on the hotspot's location but in the location where the hotspot should be. There is only one small problem with this solution. The hotspot's location is different for each cursor. This means that before reacting, the application needs to check what cursor is being used and after that calculate where it should react. This does not solve the issue completely as working like this creates dead areas that are just not clickable. However, no widget is on such a dead zone, so it is ok.

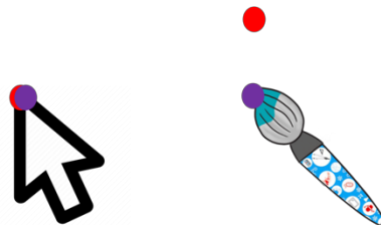


Figure 15 Hotspot issue with custom cursor. Red = Real Hotspot. Purple = Desired Hotspot

There is another issue. When a custom cursor is loaded, and the user clicks somewhere outside of the application the cursor changes back to default because it is outside of this application's scope. However, when going back to the application the cursor does not revert to the custom cursor because it thinks that the cursor did not change. This issue will be solved in the following section because it will be solved with the use of mouse events.

Mouse events

When the user clicks on the canvas, the application needs to react based on what has been pressed on. Tkinter offers easy access to tags and turns creating mouse events to a walk in the park.

At first, I used calculations to know if I clicked on a widget or not. Because it is possible to access the different widget's sizes and applied margins, it is possible to perform reverse

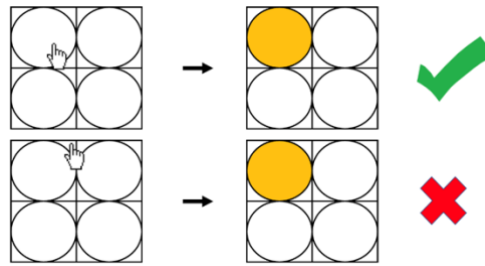


Figure 16 Click issue

calculations to get back to the widget. These calculations look and feel difficult. Furthermore, with these calculations, we are assuming that every widget is shaped like a square or like a rectangle. This means that if on a rounded rectangle you click outside a corner that has been rounded, the button will still be triggered. This is not the end of the world but makes using the application feel weird.

To fix this problem, tags are going to be used. With tags, no calculations are needed. When previous widgets have been created, a corresponding tag has been added. The widgets have been “categorised”. Tkinter has built-in functions that allow modifying multiple widgets with one command. These functions use tags. Call the function with a given tag and specify what needs to be changed. Now that change will be applied on every widget with that tag.

You can compare the tag situation with promotions in a retail shop. Sometimes during sale periods, they put a poster saying that everything with a blue sticker is 50% off. The cashiers know of that rule and when they see the blue sticker, they reduce the price by 50% without you needing to tell them about the promotion.

You may still not understand how the tags applied can help ease the creation of mouse events and that's ok because I have not told you about the “current” tag. The “current” tag is a special tag automatically managed by Tkinter. During runtime, this tag is applied on every widget located under the cursor's current position. It is also removed automatically when the cursor is no longer on them.

Mouse events are applied on widgets with the “bind()” function. So, because the widgets that the user can interact with are tagged, it is possible to attribute mouse events to tags.

For example, binding a method to the tag “field” would place that binding in the 10x16=160 draw fields. Each field would know how to react when clicked on. Unfortunately, this does not work with this implementation because of the custom cursors. Mouse events use the coordinates of the hone points and not the adapted coordinates. I had to find a solution to make this work again but with the custom cursors and the misplaced hone point.

The widgets that need mouse events have the same parent: the canvas. The idea is: When I click on canvas, click elsewhere. With elsewhere being the desired hone point. So, two mouse events are added to the canvas. One for clicking and one for dragging. This works because the mouse does not need to directly interact with the widget linked too.

Tkinter has another method that will help us. The “find_overlapping()” method. That method takes coordinates as parameters and returns every widget covering these coordinates. That method works with points and areas. Here, it will only be used with points.

To resume, when you click anywhere on the canvas, the mouse coordinates are taken. Remember the hone point is misplaced. Calculate where the hone point should be. Use “find_overlapping()” with these coordinates and store these widgets. Finally, iterate through every widget and check with the tags what action needs to be triggered.

```
262 def mouse_click_event(self, event):
263     self.window.config(cursor="")
264     self.window.config(cursor=self.curr_cur)
265     xpos = event.x + self.shift_mouse[0]
266     ypos = event.y + self.shift_mouse[1]
267     event_widgets = self.canvas.find_overlapping(xpos, ypos, xpos, ypos)
268     for widget in event_widgets:
269         if self.tags[0] in self.canvas.itemcget(widget, "tag"):
270             self.fill_field(widget)
271         elif self.tags[1] in self.canvas.itemcget(widget, "tag"):
272             self.change_color(event_widgets[0], widget)
273         elif self.tags[3] in self.canvas.itemcget(widget, "tag"):
274             self.enable_eraser()
275             break
276         elif self.tags[4] in self.canvas.itemcget(widget, "tag"):
277             self.erase_all()
278             break
279         elif self.tags[6] in self.canvas.itemcget(widget, "tag"):
280             self.confirm_send()
281             break
282         elif self.tags[7] in self.canvas.itemcget(widget, "tag"):
283             self.save()
284             break
285         elif self.tags[8] in self.canvas.itemcget(widget, "tag"):
286             self.load()
287             break
```

Figure 17 Mouse event with tagging and mouse shift

In line 263 and 264, the previously mentioned cursor issue is solved. The program is told to switch to the default cursor and revert to the custom cursor.

Following most actions is a break. This has a reason. The buttons have text, and the text is also tagged with the same tag. Without breaking out of the loop for some of these actions, the program would perform some actions twice. Sending the same picture to the robot or erasing the draw area twice does not make sense. That is why it is important to break after some actions.

Next, the different actions will be explained to you.

Changing Colour

When you click on a field of the colour palette, the brush (custom cursor) needs to change to fit that colour. The background of that colour field changes too. One must not forget to reset the previous background to default. The selected colour is remembered.

Drawing Fields

When a field is clicked, the current colour is used to fill that same field.

Before knowing about tags, more complicated methods have been created and used. It is not because a solution is difficult, that the solution is good. The program kept crashing with the previous solution which consisted of putting a new field with the current field on top of the current field instead of simply changing its fill colour.

Changing the colour of the fields (erasing inclusive) is the only action that can be executed while holding the mouse pressed. Dragging it over multiple fields will fill each of them with the current colour.

Erasing

This application has two buttons dedicated to erasing. The first button enables the eraser and allows erasing specific fields on the drawing area. The second button clears the drawing area.

Using the eraser is nothing more than filling the draw fields with the white colour. When the eraser is enabled, the cursor changes to an eraser.

The “erase all” button clears the field. Before clearing, a prompt window appears asking the user for certainty as the button press may have been an accident.

Loading and Saving

The loading button loads into the draw area a painting from a specific directory. The save button stores the current painting in that same directory. How both work will be explained in the backend chapter.

Sending

The last button sends the necessary instructions to pipette the painting to the robot. This button also creates a prompt window and an info window with the pickup code for the user.

Conclusion

Implementing an interface with Python and Tkinter is not very difficult. One just needs to know what (s)he wants. The next thing to do is discover how to code it. If the code is too complicated, it is probably not the best solution. Tkinter offers lot of different ways of implementing ideas.

Application Computational Part

In the computational part, algorithms running in the background will be explained. The interface part was more about the look-and-feel of the application.

For the methods running in the background, an additional script has been developed. This script manages everything from checking if output directories exist to executing bash scripts in the background. This script is imported by the script creating the interface. The interface calls the methods stored in here. The interface needs to share the painting and the colours of the colour palette to use some functionalities.

Libraries Used

- Pillow allows processing images with Python.
Documentation: <https://pillow.readthedocs.io/en/stable/>
- NumPy is the go-to Python library for scientific computation.
Documentation: <https://numpy.org/>
- Python modules:
 - o time. Offers some time-related functions.
Documentation: <https://docs.python.org/3/library/time.html>
 - o subprocess. Allows running other processes.
Documentation: <https://docs.python.org/3/library/subprocess.html>
 - o os. Allows manipulating the running operating system.
Documentation: <https://docs.python.org/3/library/os.html>
 - o random. Offers random number generators.
Documentation: <https://docs.python.org/3/library/random.html>

Creating painting matrix for backend

The backend will need for some functionalities that the interface shares the painting's composition. The interface does not share the fields of the draw area but instead creates a matrix with the same amount of elements as the draw area. Instead of storing fields, this matrix stores the colours used for each field. A matrix is a multi-dimensional list.



Figure 18 Creating the painting matrix

To create this matrix, an algorithm has been developed. This algorithm goes through every widget tagged with “field” and puts the colour of each field in the matrix. The resulting matrix will be sent to the script with the backend functionalities.

```
1 def create_matrix():
2     widgets = get every widget on canvas
3     matrix = empty matrix
4     row = empty list
5     count = 0
6     for widget in widgets:
7         if "field" in widget's tags:
8             count = count + 1
9             add widget's fill color to row
10            if count % number of columns in draw area == 0:
11                add row to matrix
12                empty row
13    return matrix
```

Figure 19 Pseudocode to create painting matrix

Saving and Loading

In the interface, there is one button responsible for loading previously drawn pictures into the draw area. This button will be enabled for the public. For the application to be able to load pictures, some pictures are needed. The save button saves the current picture in the directory accessed by the load button. The saved pictures can be loaded into the draw area. The save button will not be enabled for the public as children may fill the pool with ugly and/or inappropriate pictures.

For this to work, an additional directory is needed. Let's call that directory “examples”. Before doing any of both, the program needs to be sure that the “examples” directory exists. If it does not exist, the directory is created before saving. If the directory does not exist when trying to load a picture, nothing happens.

When saving, the next step is creating the path of the file. The output path looks like: “<path-to-project-root-folder>/examples/<pickup-code>.txt”. The pickup code is the same pickup code as when sending files to the robot. Next, an empty variable is defined. In this variable, the content of the output file will be written inside.

To use the save functionality, the interface needs to share the painting and the colour palette. Because the next step is going through every field and checking the colour of every field. When the colour of a field has been retrieved, the position of that colour in the colour palette is picked up next. The position of that colour is written at the end of the output file. The colour white or the colour of empty fields is not on the colour palette. For these fields, the value -1 is used.

Once every field has been checked, the output file is created and stored in the “examples” directory.

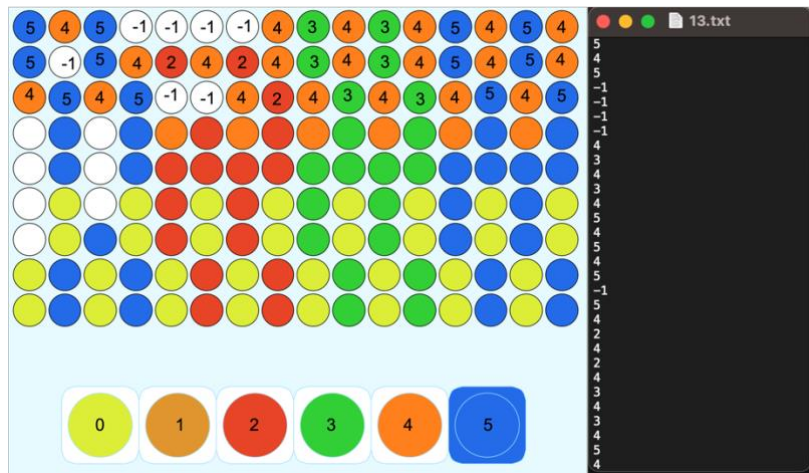


Figure 20 Saving and loading.

Loading is practically the same but in reverse. Read the .txt file and send the content to the interface. The interface then iterates through every draw field and applies the colour of the colour palette at that position.

Creating output files

The application generates a .jpg file of the painting and a .txt file with instructions on how to draw that painting.

Creating a .jpg file

The paintings size needs to be increased or else the output image is simply too small.

When the user is done painting, the painting can be sent to the robot. I have been asked to store everything that is sent to the robot. These stored paintings are not supposed to be loaded into the draw area. They are simply stored to see what has been drawn and for the case that something goes wrong with the robot.

With the Image module from Pillow, arrays created with NumPy can be converted into a .jpg file. NumPy can convert lists into arrays if conditions are met. Lists and arrays look very similar. The only difference is that lists can contain values of different types and arrays can only contain same-type values.

The painting has a dimension of 10x16. This is not big at all. So, before saving the painting into a .jpg file, an empty matrix will be created. This matrix can be imagined as a pixel grid in which each pixel is going to be filled. To make the painting bigger, a field (1x1) is added

to the matrix multiple times to get like a big 40x40 square of pixels of the same colour as the current field. The painting is no longer 10x16 but 400x640.

```
15 def create_jpg():
16     pixels = empty list
17     for row in painting:
18         row = empty list
19         for field in row:
20             repeat 40 times:
21                 add color of field to row
22             repeat 40 times:
23                 add row to pixels
24     convert pixels list to array with NumPy
25     convert array to image data with Pillow.Image
26     save image data as .jpg file in corresponding location
```

Figure 21 Pseudo code to create .jpg files of paintings

The output image will not look like the painting in the draw area. Simply because we are storing each field as square chunks and not as oval chunks.

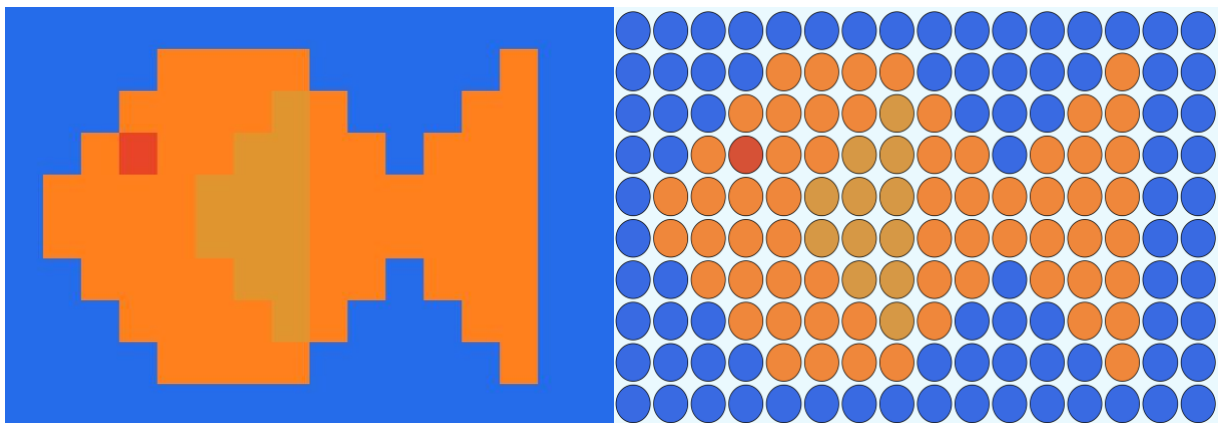


Figure 22 Comparison between output image and painting in draw area

Creating the instructions for the robot

The output .txt file is a series of instructions that the robot has been taught to understand.

```
28 def get_instructions():
29     commands = empty string
30     for color in color palette:
31         occurrences = get occurrence of color in painting
32         if occurrences > 0:
33             add fill command to commands variable
34             for field in painting:
35                 if current color and field color match:
36                     add paint command to commands variable
37     return commands
```

Figure 23 Pseudocode on how to create instructions for the robot

In Figure 23, the pseudocode shows that the instructions are colour oriented and not field oriented (line 30). This means that the robot will paint colour by colour instead of field by field.

The colour palette in the application is in a 1x6 grid but the colour palette used by the robot is 2x3. This conversion must be calculated. The robot needs to know on which row and column the current colour is.

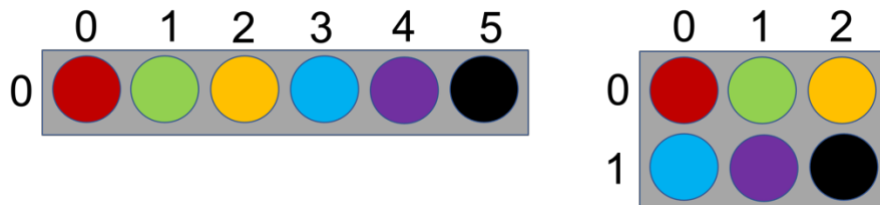


Figure 24 Colour Palette difference.

Afterwards, the number of fields filled the current colour is counted. If the colour has been used, a fill command with the column and row on which the colour can be found and the number of fields to paint is added to the instructions. Following this command, one paint command per field is added to the instructions. The resulting instructions can be stored in a .txt file.

```

1 fill 0 1 12
2 paint 2 7
3 paint 3 6
4 paint 3 7
5 paint 4 5
6 paint 4 6
7 paint 4 7
8 paint 5 5
9 paint 5 6
10 paint 5 7
11 paint 6 6
12 paint 6 7
13 paint 7 7
14 fill 0 2 1
15 paint 3 3
16 fill 1 1 61
17 paint 1 4
18 paint 1 5
19 paint 1 6
20 paint 1 7
21 paint 1 13
22 paint 2 3

```

Figure 25 Instructions to paint fish from Figure 22.

Sending files to robot

Sending the instructions to the robot cannot be done with this script alone. An external bash script is necessary.

When the send button is pressed, the painting is stored as a .jpg file and the instructions are stored in a .txt file with the two previous algorithms. The robot does not need to receive the painting. It only needs the instructions to reconstruct the same painting.

Sending the instructions to the robot is maybe the trickiest part. Different things with Python were tried but none worked. However, sending files over to the robot with the Visual

Studio Code was possible. Next, I tried just sending things over to the robot using the command line because Python can execute with the subprocess library command line commands.

Sending files over to the robot ended up working. To send files over the “scp” command is used. “scp” is an acronym for “Secure Copy Protocol”. This protocol is used to send files between Unix systems. The Mac and the Ev3Brick both operate on a Unix system. The “scp” command uses the IP address of the robot to send files over to it. The system will prompt the robot’s password to make sure that we are allowed to send it files. This turns out to be a problem. In Python, subprocess can execute the “scp” command but subprocess does not work well with password prompts.

The next thing to look for is a way of bypassing the password prompt. For this the “sshpass” command is used. SSH stands for “Secure SHell”. With SSH, one can have remote access to a different machine. “sshpass” connects to a remote machine and bypasses the password prompt step. The password prompt is not skipped. The password is given to the command so that it can fill the password prompt without the user noticing it.

It is dangerous to use “sshpass” as you are turning your password vulnerable. Here, it is not as dangerous as we are only connecting to an Ev3Brick that can only move a robot. Additionally, the device trying to connect to the robot must be connected by cable to the robot. No Bluetooth and Wi-Fi is enabled on the robot.

In the [ev3dev documentation](#), it is explained how to connect to the robot per SSH. The IP address and password of the robot can be found here. Sometimes a device connected by cable to the robot cannot find the robot with its IP address. When that happens, the IPv6 connections need to be turned off and the IPv4 addresses need to be changed to “Link Local Only”.

To send a file to the robot a line like this is used:

```
sshpass -p "maker" scp /path/to/file robot@ev3dev.local:/home/robot/queue/
```

In this line, the main command is “sshpass”. Its flag “-p” stands for password and the robot’s password “maker” follows right after. Next is the command we want to use to send the file to the robot. The “scp” command. This command takes two arguments. The first argument is the file being sent. The second argument is the file’s destination. The destination is made of two parts. The first part is the robot’s IP address. The second part is the destination location in the robot’s file system. Both parts are separated by a “:”.

This is only one line and Python can execute one line with subprocess. At the moment, I did not know if something will need to be added or not. So, I created a bash script. The content of this bash script is the “sshpass” command explained. Running a bash script only takes one

line in the command line. With this external bash script more than only one command line can be executed by Python at the same time.

The file to send over is not always the same. The bash script needs to be dynamic. For this reason, the bash script expects one argument, the path of the file to be sent.

The Python code to execute this script can look like this:

```
subprocess.run(["./bash_scripts/send.sh /path/to/file"], shell=True, check=False)
```

One last change must be applied on the script for it to take the first argument. The “/path/to/file” must be replaced with “\$1”. Finally, the sshpass command looks like this:

```
sshpass -p "maker" scp $1 robot@ev3dev.local:/home/robot/queue/
```

While the robot is receiving these files, the robot will be running its own script. The robot is not intended to run things in parallel. Sending files to the robot when it is not running takes no time. However, when it is running, it may take up to 45 seconds.

Robot

The pipetting robot only runs on program. This program tells the robot when and how to move. The robot's script uses the os and the pybricks libraries. The pybricks libraries are specialised for the Ev3Brick. It offers lots of functionalities that will help manage the robot.

Before doing anything else, the connection to the brick and to each motor and sensor must be established.

```
39      # ev3brick, motors, and sensors
40      self.ev3 = EV3Brick() # ev3brick
41      self.motor_a = Motor(Port.A) # motor moving whole body (vertical movement)
42      self.motor_b = Motor(Port.B) # motor filling/emptying pipette
43      self.motor_c = Motor(Port.C) # motor moving upper body (horizontal movement)
44      self.motor_d = Motor(Port.D) # motor moving pipette up/down
45      self.gear_s1 = ColorSensor(Port.S1) # color sensor for motor b and d
46      self.gear_s2 = TouchSensor(Port.S2) # touch sensor for motor a
47      self.gear_s4 = TouchSensor(Port.S4) # touch sensor for motor c
48
```

Figure 26 Establishing connection between pipetting robot hardware and code

Movement

In the following subchapter, explanations on how the robot moves will follow.

Defining the robot's starting point

The robot has no conscience of its environment and its surroundings. When the robot is launched, the robot defines its current state as the default state or starting point. This is not good. The robot needs to start always in the same position. If you refer to Figure 5, you will see that motors work together with sensors.

The sensors are lined up so that every one of them gets triggered when the robot is far out on the upper right corner. The next step is moving each motor towards the direction that will trigger the corresponding sensor. The motors movement must be stopped when the corresponding sensor is triggered.

The last step is figure out in what order the motors need to be moved. The colour palette and the paper support will be lying inside the robot's field of activity. Blindly moving the motors may cause the pipette to get stuck somewhere. The first motor that must be moved is the one lifting and sinking the pipette. The remaining order is not important anymore as the pipette is the only element that can get stuck or collide somewhere.

```
# Motor D
self.motor_d.run(-self.speed)
while self.gear_s1.color() != Color.YELLOW:
    wait(1)
self.motor_d.hold()
self.motor_d.reset_angle(0)
self.motor_d.run_target(self.speed, 40)
self.motor_d.reset_angle(0)
```

Figure 27 Moving a motor to its start position

In Figure 27, the motor lifting and sinking the pipette is being moved to its new starting point. The robot consists of motors that spin. The values given to movement functions are all measured in degrees.

The “run()” function tells the motor to spin with a constant speed measured in degrees per second. The program is “paused” until the colour yellow is detected with the colour sensor. When the colour yellow has been seen, the motor is told to hold. The motors have three different types of braking. Holding forces the motor to brake in that instant on that position. The other braking styles, let the motor brake due to friction. The “reset_angle()” function defines the motor's current position has default or starting position. Because the pipette does not need to go all the way back to prevent collision, the pipette is lowered a bit. The “run_target()” method needs to know how fast the motor needs to spin. The second value is not how many degrees are being done but the destination angle of the motor. If the motor is currently at 0 degrees and you ask it to go to 40 degrees, the motor will spin for 40 degrees. If it starts at 50 degrees, the motor will spin for 10 degrees backwards.

The other motors work the same. Move them towards their sensor, stop when sensor is triggered, and remember new starting position.

Move robot to specific position

The robot uses the row and columns to calculate where it needs to go.

In the painting application, the fields of the drawing grid and the colour palette are often referred via their column and row. The robot will do the same. Some additional values need to be figured out.

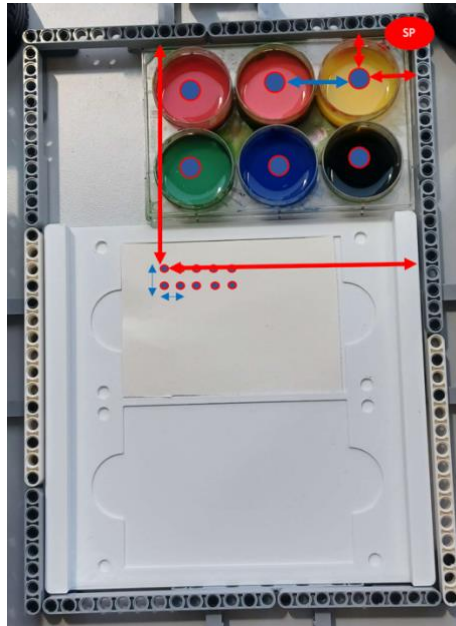


Figure 28 Values to discover in robot's field of activity

In Figure 28, you can see where the robot's starting point (red oval) is and what values need to be figured out. The red arrows are the distance between the starting point and the colour palette or the draw area. For the colour palette, that distance is 0. For the draw area, some values have been tested, and one that worked was kept. The values used in this implementation are not the only working values.

The next values are the degrees that the robot's motor must spin to cover the distance displayed with the blue arrows. Here again, the values used are not the only values that do the job. The columns and rows of fields will be multiplied with these values. The result of this multiplication returns how many degrees the different motors must spin to go to a specific field.

Additionally, you may have noticed that the robot moves from the right to the left. So, if the robot wants to go to the colour yellow at position (0;2) and the robot receives (0;2) as destination, the robot will go to the colour red at position (0;0). This happens because our grid systems work from left to right and not right to left. This issue only occurs with the columns.

The rows work perfectly fine because the direction is the same. To solve this, the column coordinate needs to be mirrored for the robot to go the right place.

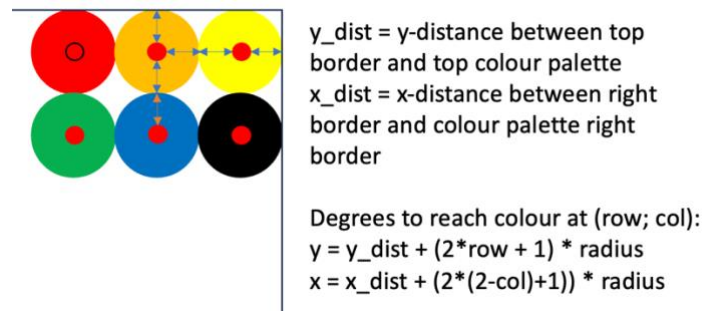


Figure 29 calculations to reach specific field

Picking and releasing colour

The robot can pick enough colour to paint a painting in one go. Most problems with colour are problems that cannot be solved with computer science.

To pick up colour, go to the desired colour and lower the pipette. The pipette's tip should be in the colour. Once inside, use the syringe's motor to fill the pipette with colour.

Moving the motors is no problem. To find the adequate values on how much colour should be released per field and how many fields can be coloured in one go, practical tests that will be explained later were performed. Testing teaches us that the pipette can take enough colour to fill 600 fields. Now, it is also known how many degrees the motor needs to spin to colour one field.

So, nothing new is used. To pick up or release colour, go to a field and spin the syringe's motor.

In this implementation, I remember what colour was used last because I prefer filling a bit more colour than necessary for when something goes wrong. By adding this, I must not forget to empty the pipette before picking up a new colour.

Listening & Reacting

The robot needs to keep a queue with paintings to be done. When the queue is empty it should wait.

```
39 def run():
40     if paintings in queue:
41         code = first painting's code
42     else
43         code = shared code
44     define start position
45     go to start position
46     repeat until stopped:
47         if expected_code.txt in queue:
48             expected_code = expected_code + 1
49             remember instructions
50             if second painting:
51                 update display
52                 start painting
53                 update display
55 def painting():
56     take instructions
57     for each instruction:
58         read instruction
59         if "fill" in instruction:
60             empty pipette
61             go to color field at ([col, row])
62             fill for enough fields
63         if "paint" in command:
64             go to draw field at (col, row)
65             release colour for one field
66     delete instructions and .txt files
67     empty pipette
68     go to start position
```

Figure 30 Pseudocode of robot

Launch

The robot is best launched together with the painting application. To launch this project, an additional bash script was written. First, the script checks in the output folder what the next code will be. Next, the script checks the operating system and opens the right version. Finally, the script connects to the robot with “sshpass” and shares with the robot the next code and launches the application.

Boot

When the robot is booted, the robot first checks its directory where the paintings that need to be “printed” are located. If there are still paintings, the robot will start the painting named after the lowest code. If there are no paintings, the robot waits for the code supposed to be shared with it through the bash script.

After knowing what code to expect next, the robot moves the motors to the defined starting point and starts observing the queue directory.

Queue

The robot watches the queue until stopped.

While waiting the robot displays on the screen of the Ev3Brick what code it is waiting for. When a .txt file named after the expected code is in the queue directory, the robot remembers the instructions and waits for a second painting after updating the expected code. The display is also updated. The robot is programmed to paint two pictures at the same time. Painting two

pictures instead of only one, helps the person that will need to keep an eye on the robot during public events.

When two pictures are ready, the display is updated and prompts a button press of the person watching the robot. The robot asks that same person if the papers are set and if enough colour is in the colour palette. After confirming, the robot starts painting.

The robot can receive new paintings while painting.

Drawing

To draw, the robot reads the instructions written by the painting application and follows them step by step. The instruction says what command to use on which row and on which column. If the instruction asks for the fill command, the instruction even says how many fields will need to be filled.

When done painting, the current instructions are forgotten and deleted from the system. The pipette is emptied, and the robot moves to its starting position.

Explanation Application

This chapter is going to be a short one. The explanation application is supposed to run on a different screen or computer than the painting application next to the pipetting robot. This application just plays an animation showing and explaining how the robot proceeds to paint the different pictures. The instructions displayed on the right side of the screen are the actual instructions that would be sent to the robot to paint that same picture.

This application is longer than the painting application, but nothing new is used in this application. Even though this one looks a bit different, everything used here has already been used in either the painting application or in the robot's script.

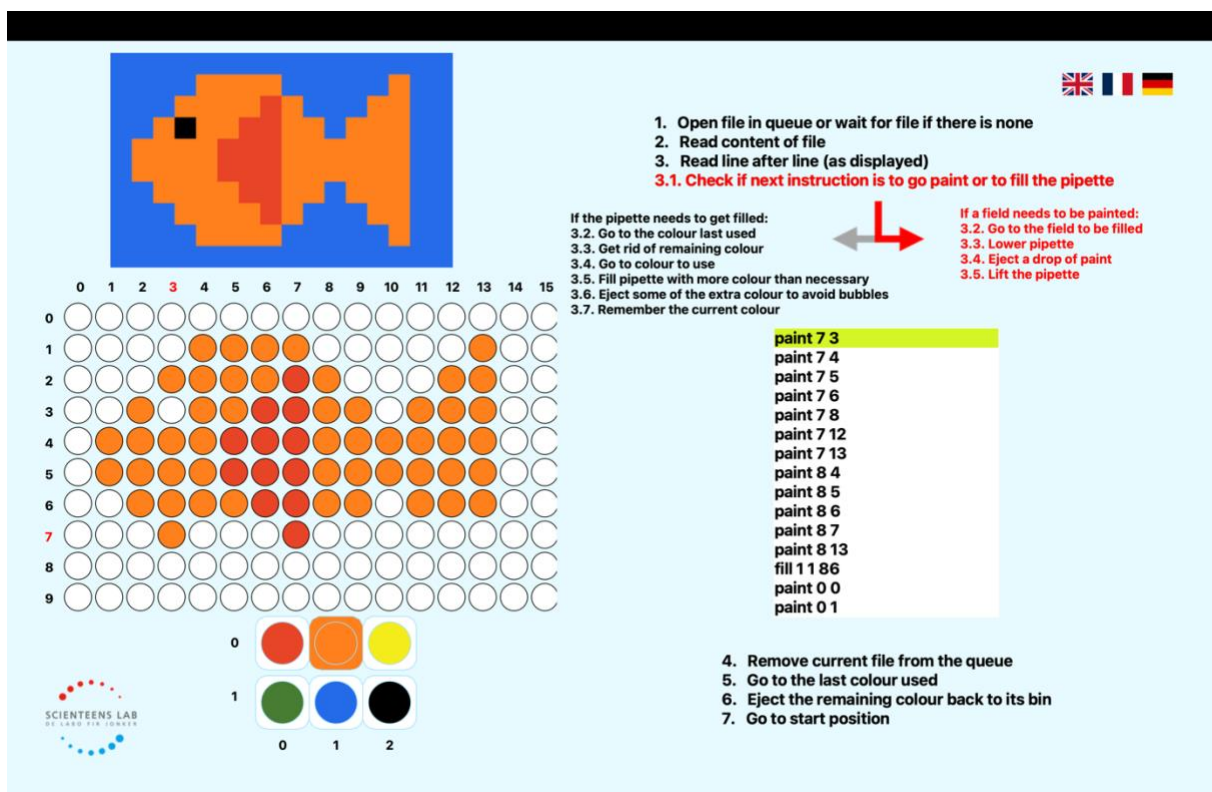


Figure 31 Explanation Application

Testing and Validation

For this project, I preferred testing with real user feedback over writing unit tests. For the moment, only Scienteens Lab people have tested and played with the application. We are still missing user feedback from our target audience. The computer science part of this project has been tested and everything seems to work.

For the rest of the project, it is a bit more complicated, as other factors have an important role in this project. The robot is made of Legos, and this causes it to not work the same every time it runs as the Legos are not the sturdiest support. The paper also needs to be tested. New paper has been ordered and it has not arrived yet.

Finally, feedback from users, preferably high school students and teachers as they are the Scienteens Lab's target, would be nice. Before getting feedback from them, the robot should be ready, and the physical material should work too.

The different approaches of testing will be described in the next two subchapters. The first one focuses on what tests have been accomplished and the second one focuses on what still needs to be tested.

Accomplished

For the painting application, no unit tests have been written as most code that is written for it affects visual elements. This means that you either see or feel the changes or not.

This does not mean that nothing was done either. I like to compare what I did to Black Box Testing. Black Box Testing is testing a program without knowing how it was built. Most higher-ups of the Scienceteens Lab have minor programming knowledge if any. So, this kind of approach was perfect.

I developed a new feature and went to them and asked for them to try it out. There were times where things did need to get changed. There were times where the idea was better than the actual implementation and vice-versa. And there were times where the feature was working from the beginning. This kind of interaction was often what occurred during short briefings. Sometimes there was someone from another department or office that did not know about the project and just tried it. The biggest changes came from this kind of interaction.

For this project, I think that this kind of testing is perfectly fine because at the end of the day, this project will be used by people that have no programming knowledge. So, getting feedback from them on what needs to be specified or changed is important. Scienceteens Lab wants to incite students to get into the different domains of science.

For the robot, the specs of the robot needed to be tested. This is more like White Box Testing. White Box Testing is testing a program knowing how it was built and works. How fast can the motors spin? How much colour can the pipette take? How long can the Ev3Brick work? What is the paper's best setup? What type of colour works best? Many questions need to be answered. To answer most questions, experimentations were done with the robot.

Observing how far the robot moved with a certain number of degrees moves. How deep can the pipette go. And more observations of this kind were done. This took some time and is repeated until everyone is satisfied with the result. These tests are not over yet. They are still going on in hope of finding a better combination of values that produces an even better result.

These tests are not only important to find out what works but also to find out what does not work. During testing, the robot showed that the more it moves towards its left or right border the more imprecise it gets. Because of this, the result paintings need to be kept small and centred.

The tests performed consisted of changing parameters in the code and then observe the robot's movement. Just by observing what happens in the code, it is impossible to guess how

the robot is moving. The parameters needed to be adapted after each run, until good parameters have been found.

If there was a conversion between degrees and meters, I could calculate the number of degrees necessary. So, I let one of the robot's motors turn for a 100 degrees and I measured with a ruler how many meters the robot just did. Because the robot is made of Legos, the robot is not consistent in its movement. This means I could not simply trust that conversion rule but had to confirm this afterwards. The conversion rule was not useless. It still gave me an approximate idea with what number of degrees to start testing when I wanted the robot to move a specific distance. That number had to be then adapted to the robot's inconsistency.

Because of the robot's inconsistency, we must not expect the robot to always hit the same spot. However, the inconsistency is not important enough to destroy the painting being pipetted. This inconsistency may result in one droplet being a bit off target or the painting being placed more left than right. Still, the painting ends on the piece of paper. The inconsistency has been taken into account and the different parameters decided so that they work with this inconsistency.

Another question created during testing is how the drying will be handled. The paper's absorption was until now not good enough to let the pictures just dry by themselves. Now, waiting until the painting is dry takes around 40 minutes. This must be avoided, and the drying must be accelerated by a lot. Either by finding paper with better absorption, changing the colour, or installing a system that dries the painting. This is not directly affiliated to computer science but still something that must not be neglected. The last thing tried was a hair dryer.

Why not leave the painting dry naturally? The robot will be used by the public during exhibitions. Most people will not want to wait 40 minutes for their pictures. The hair dryer decreases the drying time by a lot but requires that the person using it stays focused because not remaining consistent may cause the droplets to disperse and destroy the grid look of the painting.

Not Done Yet

There is not much left to be done. The paper has been ordered and ideas for drying are being gathered. Once this is done, feedback collecting can start. The feedback form has already been realised. The targets of the Scienceteens Lab are high school students. So, the ideal people to ask for testing would be high school students and maybe high school teachers.

They will be asked to use the painting application and observe the robot pipetting their artwork. After using the robot, they will be asked to fill a form so that I can observe what works and what does not work. The results can then be discussed with other Scienceteens Lab members to decide how to improve this project.

The form for the painting application has already been done. First, common questions about the person are asked to learn about who is being dealt with. Age, gender, and occupation are asked. Next, the person must tell us if they are interested in biology and/or computer science. Finally, the last questions ask if the person can guess what the robot will do or how it works. For this, the person will be allowed to look at the robot and the painting application without being allowed to use it. With this information, we can observe whether the application is appealing or deterring by getting the information on how much can be understood without having to interact with the robot. After having answered to these questions, the person is allowed to use the robot.

After having used the robot, additional questions are asked. First, the person is asked if its interest towards science has increased. Next, the subject is asked for its global opinion on the project. This question could be asked in the end, but I am afraid to bias the person into only thinking about what has been asked. Following this, I ask if there are things that the person would feel more comfortable with being told about before testing the robot. Finally, the form ends with simple questions to know whether the robot and its painting is a success or not.

Conclusion

The project's realisation is done. Physical problems still need to be solved.

The robot's computer science part is done. The applications run and the robot pipettes. Remaining work is testing the paper and ask for feedback. These things could not be done yet because we did not know how things would work out at the beginning. Figuring out what material works best is not that easy because there are always details people do not know about. At the beginning, I did not think that material would have such an impact on the project. Now, new material has been ordered and we are waiting for it.

The goal of this project was building a project that may interest high school students. Doing something that looks complicated and implement it in a simple way is the best way to hook new people in my opinion. I cannot tell for sure if I succeeded as I am waiting for feedback, but I think I did a good job to get that result. When I look back at the code, I am surprised how simple this looks. The reality is that behind the simplest things lots of time was spent to get to that simple solution. This just shows that what turns computer science complicated is often the person behind the screen. Do not think of complicated things and coding stays simple.

Just think of what needs to happen, maybe write the algorithm down, and you have already done most of the work. Next, is turning everything comprehensible to your computer which is not more complicated than trying to speak with a foreigner. Use a dictionary and you will get the words you are looking for. Coding is the same, especially with robots.

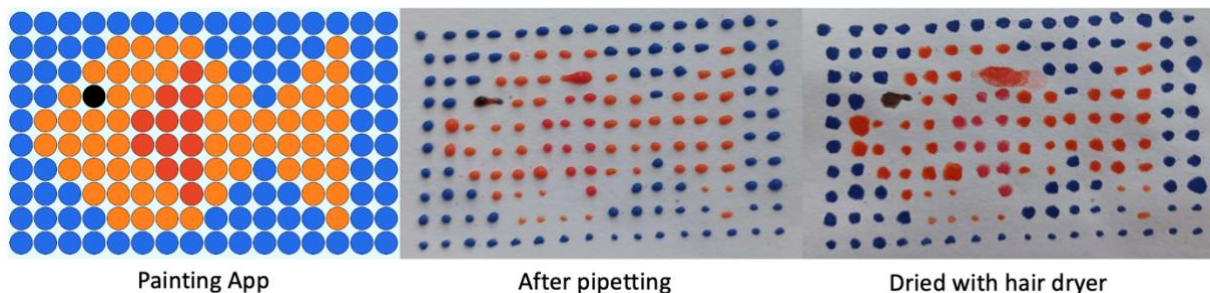


Figure 32 Artwork from PC to paper

Improvements for the future

The robot can still benefit from many ideas.

The Raspbian version of both applications still needs to be updated. It would be great to find a solution to run this program on Windows.

The robot is very inconsistent because it's made of Legos. Everything that adds consistency to the robot is welcome. Now, there is sometimes a piece that jumps out. It does not stop the robot from running unless it is missing at the start. Other pieces of the robot have similar situations in which they do not stop the robot, but it is exhausting to have to watch out for these small inconveniences.

Another idea would be to setup a database to store the pictures. Keeping them on the project folder works too but at some point, there will be too many pictures.

Ideally, the robot would get one more sensor to be able to recognise colours. This way the robot could be programmed to act with colours instead of acting with whatever colour is on position (x;y).

However, adding too much is also not the best idea as this project should remain simple. It does its job already. It just needs a bit of fine-tuning.

Some words about the internship

The internship did not go as planned. At first, at least one more project was planned to be done in the meantime. No one expected the robot to have this many flaws to fix. There were many things that did not go as planned or that did not work at all. New ideas were necessary and old ideas were dropped.

However, because the pipetting robot is not fully done yet does not mean that I did not work on other projects. I ended up participating in one or another additional project when it was urgent. These projects are not shown in this paper because prior context would be missing, and it would not do any good to rush the explanations of the other projects as they are a bit more complicated.

One involved trying to upload an OpenCV YOLO (You Only Look Once) Python Flask project on WordPress. In this sentence alone are already so many terms that need explanations. To keep it short, the project is about hosting a face detection application on the Internet. There is too much behind it to just be a small part of this paper.

In another project, I helped Tetiana working on the website developed for the Scienseens Lab. I was mainly working on translations. So, here it's rather the contrary. Behind translating is no research unless you want me to write you how I used online dictionaries to translate from English to French or German.

And yet, these projects still taught me things that could be applied on any kind of project. Working with Tetiana increased my understanding on the importance of good communication. Good communication avoids wasting time.

The face detection application showed me how important it is to keep your code tidy and commented. The face detection application was already developed but I still need to understand it. There are still things on that application I cannot understand, and I can imagine that during team projects these kind of situation can be fatal.

Lastly, I want to add some words about my student job with the Scienseens Lab. I should not write about it here, but it is too important for me to not write about it. I helped hosting computer science workshops for high school students. During these, I got in contact with high school students, the main target of the Scienseens Lab and of the pipetting robot. These workshops helped me better understand how students interact with computers and the different applications used during the workshops. Many design decisions of the pipetting robot and other projects are based on the experience gathered during these workshops.

Doing the project from scratch

In this short chapter, I want to tackle some points that I should consider when doing a project like this.

First, see what the client expects from you. Next, read the documentation and maybe do mock-ups to compare with the client. Creating mock-ups would save me tons of time as they are easier to do with specialised software than coding it. When one mock-up has been picked, the implementation can start. This is how I would tackle this kind of project now at least for the different interfaces.

For the robot, it is difficult to say if I would do something differently. I think I would proceed the same way with the robot. Learn about the robot and adapt. Repeat until satisfied.

The following statement is not only for this project but for all projects. I need to take more notes of what I am doing, what I learned, and what I am planning to do. This is not just for me. This benefits the whole team as it becomes easier to share with others your progress. Discussions become more meaningful too as you can skip over topics or facts that were noted down.

To summarise, I wish I spent more time analysing and designing before starting to implement the project.

Appreciation words

I would like to thank everyone who participated in the project and this thesis. My thanks go out to every member of the Scienceteens Lab that stopped by the robot to give feedback or just ask what the robot was about. I would also like to especially thank Dr. John Elisabeth, Poussin Régine, and HR for organising this internship, Dr. Philip Goergen for the main project.

Dr. Goergen Philip, Dr. John Elisabeth, Dr. Krüger Maren, Melnychuk Tetiana, and Poussin Régine were available every day for all kinds of questions concerning any of the projects and this paper. They also gave the most feedback about the robot. Thank you for all your help.

Next, I would like to thank Prof. Dr. Denis Zampunieris for agreeing to be my supervisor and for all the precious advice given during the internship.

I would like to also thank Prof. Dr. Volker Müller for all his hard work and for giving us this opportunity.

I know that I do not know everyone behind the realisation of this internship. Sorry for this but please accept my thanks for allowing this to happen. I am grateful for everything I have learnt so far be it during the internship or during the bachelor's degree.

Table of Figures

FIGURE 1 SCIENTEENS LOGO	7
FIGURE 2 IMAGINATION OF HOW THE PROGRAM WILL RUN	8
FIGURE 3 PIPETTING ROBOT.....	8
FIGURE 4 FRONT AND TOP VIEW OF THE PIPETTING ROBOT	11
FIGURE 5 ROBOT'S MOTORS (CIRCLES), SENSORS (RECTANGLES), AND MOVEMENT DIRECTIONS (ARROWS)	12
FIGURE 6 EV3BRICK	13
FIGURE 7 INITIAL DESIGN IDEA.....	15
FIGURE 8 DIFFERENT DESIGNS OF THE APPLICATION	20
FIGURE 9 BASIC TKINTER APPLICATION.....	21
FIGURE 10 SETTING UP THE APPLICATION WINDOW AND ITS CANVAS	22
FIGURE 11 " __INIT__ " METHOD BEFORE CREATING DRAWING FIELDS AND COLOUR PALETTE	23
FIGURE 12 PADDING IN THE DRAWING FIELDS.....	24
FIGURE 13 "CREATE_ART_AREA" METHOD.....	25
FIGURE 14 FINAL UI APPLICATION DESIGN	26
FIGURE 15 HOTSPOT ISSUE WITH CUSTOM CURSOR. RED = REAL HOTSPOT. PURPLE = DESIRED HOTSPOT	27
FIGURE 16 CLICK ISSUE	28
FIGURE 17 MOUSE EVENT WITH TAGGING AND MOUSE SHIFT	29
FIGURE 18 CREATING THE PAINTING MATRIX.....	32
FIGURE 19 PSEUDOCODE TO CREATE PAINTING MATRIX.....	33
FIGURE 20 SAVING AND LOADING.	34
FIGURE 21 PSEUDO CODE TO CREATE .JPG FILES OF PAINTINGS.....	35
FIGURE 22 COMPARISON BETWEEN OUTPUT IMAGE AND PAINTING IN DRAW AREA.....	35
FIGURE 23 PSEUDOCODE ON HOW TO CREATE INSTRUCTIONS FOR THE ROBOT	35
FIGURE 24 COLOUR PALETTE DIFFERENCE.....	36
FIGURE 25 INSTRUCTIONS TO PAINT FISH FROM FIGURE 22.....	36
FIGURE 26 ESTABLISHING CONNECTION BETWEEN PIPETTING ROBOT HARDWARE AND CODE	39
FIGURE 27 MOVING A MOTOR TO ITS START POSITION.....	40
FIGURE 28 VALUES TO DISCOVER IN ROBOT'S FIELD OF ACTIVITY	41
FIGURE 29 CALCULATIONS TO REACH SPECIFIC FIELD	42
FIGURE 30 PSEUDOCODE OF ROBOT.....	43
FIGURE 31 EXPLANATION APPLICATION	45
FIGURE 32 ARTWORK FROM PC TO PAPER	50

Bibliography

- [1] U. du Luxembourg, 'Scienseens Lab', *University of Luxembourg*. https://www.uni.lu/lcsb/scienseens_lab (accessed Mar. 19, 2023).
- [2] 'Bluetooth Pan is missing on new Macbook P... - Apple Community'. <https://discussions.apple.com/thread/253312557> (accessed Mar. 20, 2023).
- [3] 'Installation — ev3-micropython 2.0.0 documentation'. <https://pybricks.com/ev3-micropython/startinstall.html> (accessed Mar. 19, 2023).
- [4] 'ev3dev Home'. <https://www.ev3dev.org/> (accessed Mar. 19, 2023).
- [5] SneakyTurtle, 'Answer to "How to make a tkinter canvas rectangle with rounded corners?"', *Stack Overflow*, May 21, 2017. <https://stackoverflow.com/a/44100075> (accessed Mar. 23, 2023).
- [6] 'Getting started with LEGO® MINDSTORMS Education EV3 MicroPython — ev3-micropython 2.0.0 documentation'. <https://pybricks.com/ev3-micropython/> (accessed Apr. 26, 2023).
- [7] 'LEGO® MINDSTORMS® EV3 MicroPython - Visual Studio Marketplace'. <https://marketplace.visualstudio.com/items?itemName=lego-education.ev3-micropython> (accessed Apr. 26, 2023).
- [8] 'MINDSTORMS EV3 Support | Everything You Need', *LEGO® Education*. <https://education.lego.com/en-us/product-resources/mindstorms-ev3/teacher-resources/python-for-ev3/> (accessed Apr. 26, 2023).
- [9] R. P. Ltd, 'Operating system images', *Raspberry Pi*. <https://www.raspberrypi.com/software/operating-systems/> (accessed Apr. 26, 2023).
- [10] 'Pillow', *Pillow (PIL Fork)*. <https://pillow.readthedocs.io/en/stable/index.html> (accessed Apr. 26, 2023).
- [11] R. P. Ltd, 'Raspberry Pi', *Raspberry Pi*. <https://www.raspberrypi.com/> (accessed Apr. 26, 2023).
- [12] 'SonarLint - Visual Studio Marketplace'. <https://marketplace.visualstudio.com/items?itemName=SonarSource.sonarlint-vscode> (accessed Apr. 26, 2023).
- [13] 'tkinter — Python interface to Tcl/Tk', *Python documentation*. <https://docs.python.org/3/library/tkinter.html> (accessed Apr. 26, 2023).
- [14] 'Visual Studio Code - Code Editing. Redefined'. <https://code.visualstudio.com/> (accessed Apr. 26, 2023).
- [15] 'Welcome to micropython-usermod's documentation! — micropython-usermod 1.622 documentation'. <https://micropython-usermod.readthedocs.io/en/latest/index.html> (accessed Mar. 19, 2023).
- [16] 'Welcome to Python.org', *Python.org*, Apr. 13, 2023. <https://www.python.org/> (accessed Apr. 26, 2023).

Appendix – Project Reports

The following pages will contain the different reports done throughout the internship.

University of Luxembourg
Faculty of Science, Technology and Medicine
Bachelor in Applied Information Technology

Work done during the current period:

I started with the pipetting robot. An application window for the user has been created. The application is almost done. Communication between the application and the robot has been established. The robot reacts now when the application communicates with him. The robot has also been programmed. It can move and replicate the pointing done in the application. The second project has been discussed. The second project is implementing an AI that identifies emotions into a WordPress webpage. The AI application has been done by students from other schools and the code is GitHub. I am now trying to understand the code and reading into the resources used to implement it on a webpage. I also started writing the thesis.

Foreseen work for the following period:

For the pipetting robot, the robot's movement needs to be fine-tuned. For the second project, I will try putting the AI program into a JavaScript program. Inserting a JavaScript program into a WordPress webpage is apparently easier than inserting a Python script. I am not 100% sure as I am still reading into it.

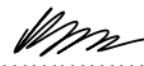
Date:

13/08/2023

12/03/2023

Signatures:


(Local responsible)


(Student)

University of Luxembourg
Faculty of Science, Technology and Medicine
Bachelor in Applied Information Technology

Work done during the current period:

- Continued working on pipetting robot
- Interface for printing application has been improved
- Adapted program to run on Raspian
- Created an auto-run and auto-installation script.
- Started working on face detection program
- Continued writing the thesis
- Running tests on robot

Foreseen work for the following period:

- Continue running tests on robot
- Test robot with real users
- Continue working on face detection app
- Continue working on thesis

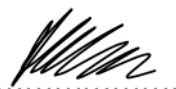
Date:

03/04/23

02/09/23

Signatures:


(Local responsible)


(Student)

University of Luxembourg
Faculty of Science, Technology and Medicine
Bachelor in Applied Information Technology

Work done during the current period:

- Read quercus.js documentation
- Continued testing with pipetting robot
- Continued writing on thesis
- Started working on additional software for pipetting robot
- read into python flask and how to use it to create REST applications

Foreseen work for the following period:

- Continue writing thesis
- continue working on new software for pipetting robot
- adapt new version for Raspbian
- upload flask application into Wordpress or similar host system

Date:

21/5/23

28/04/23

Signatures:


(Local responsible)



(Student)

University of Luxembourg
Faculty of Science, Technology and Medicine
Bachelor in Applied Information Technology

Work done during the current period:

- helped translating TestIt
- changed robot to pipette 2 drawings at a time instead of only 1
- updated the resphion version
- finished the explanation application
 - ↳ added explanation + translation
- continued improving the thesis
- fixed mouse issues in application
- started cleaning project

Foreseen work for the following period:

- cleaning the project (commenting & deleting unnecessary stuff)
in order to just write about important things on the thesis
- continue testing the robot to try and get better results
- finish writing thesis

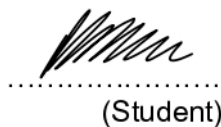
Date:

22/05

21/05

Signatures:


(Local responsible)


(Student)